DS8001 - Design of Algorithms and Programming for Massive Data

Course Project Guidelines

Proposal Due November 5 (Thursday), 2026, 6:00PM ET

Final Submission Due December 7 (Monday), 2026, 6:00PM ET

Late submissions will not be accepted!

The main objective of the course project is to perform a detailed computational/empirical analysis of a practically relevant problem using algorithmic concepts learned in this course, and to carefully document the methodology, experiments, and conclusions in a technical report. You are required to identify a suitable topic for your project, and all project topics must be approved by me. I may be able to provide early feedback on project topics if you reach out in a timely manner. Projects may be completed individually or in groups of two.

Students must submit a one-page project proposal (PDF) by November 5, 6:00 p.m. (ET). In your proposal, clearly specify which project path you intend to pursue: Empirical Analysis, Problem Solving, or Replication (see below). The proposal should concisely include: (i) the problem and the main question(s) you plan to investigate; (ii) the main algorithms/methods and what you expect to implement yourselves; (iii) the dataset or problem instances you plan to use or generate; (iv) the main comparisons, metrics, and experiments you plan to conduct; and (v) the most relevant references using a standard academic citation format. For instance, if you plan to replicate/extend a previously published paper, first summarize the main idea of the paper, and then explain which results you plan to replicate and what additional experiments, modifications, or extensions you plan to investigate.

Regardless of the project path, the project must contain a **substantive algorithmic component**. The report should explain the algorithms studied, why they are appropriate for the problem, and, where applicable, their computational complexity, theoretical properties (e.g., correctness, approximation guarantees, or convergence properties), and practical trade-offs. The project should be organized around one or more clear algorithmic questions rather than around applying a large collection of methods. A careful study of a small number of well-motivated algorithms is preferable to a shallow comparison of many methods. Simply applying several off-the-shelf algorithms and reporting their outputs might not be sufficient; a more in-depth analysis of a few solution algorithms would be preferrable.

You **must** cite any references (texts, papers, websites, datasets, software, or code repositories) that materially help you develop the project. Routine programming look-ups such as checking a Python function name do not require citation; however, you cannot copy, adapt, or use someone else's code without clearly identifying and citing the source. Externally sourced code does not count as your own implementation.

Each student or group should submit the final project report and code package by December 7, 6:00 p.m. (ET).

Here are some potential paths you can pursue for the project:

- *Empirical analysis-focused project*: Identify a practically relevant dataset or computational problem (e.g., using data from repositories such as Kaggle or UCI) and perform a detailed empirical study of a carefully selected set of algorithms. The emphasis should be on an *algorithmic question* and on depth rather than simply the number of algorithms. You should motivate the selected algorithms, explain the relevant algorithmic differences, and investigate performance measures such as solution/prediction quality, running time, memory use, and scalability, as appropriate for the problem. A generic comparison of several machine-learning models based only on predictive accuracy is not sufficient unless the project contains a clear algorithmic focus. E.g., see recommendation systems datasets in Kaggle (link).

- *Problem solving-focused project*: Identify a *suitable* problem for which alternative algorithmic approaches can be applied. While you are not restricted to these, various practical problems have been discussed during the course. For instance, you can consider the Set Cover problem and compare approaches such as: (1) an exact mathematical programming formulation solved using an optimization library such as SciPy or Gurobi, (2) the classical greedy approximation algorithm, and (3) another justified algorithmic approach, such as a local-search, randomized, or metaheuristic method. Conduct

a detailed numerical analysis on a wide variety of problem instances with different sizes and characteristics. Compare not only solution quality but also computational effort and scaling behavior, and relate the empirical results to known theoretical properties when relevant; for example, the approximation guarantee of the greedy algorithm for Set Cover. You are not expected to include every possible approach—a focused and well-designed comparison is preferable.

- *Replication-focused project*: Pick a research paper that includes computational work on algorithmic concepts relevant to this course. Reproduce a meaningful subset of the computational results, explain any discrepancies between your results and those reported in the paper, and, where feasible, investigate an extension, modification, or additional experiment. The project should demonstrate that you understand and can implement a substantive part of the algorithmic methodology rather than only execute an existing codebase. Relevant papers can be found at Google Scholar and arXiv. The papers below are examples rather than an exhaustive list; some are classic papers, and you are expected to identify relevant recent literature as part of your literature review. Here are some examples:
  - Deshpande and Karypis (2004): Item-based top-n recommendation algorithms (link)
  - Sarwar et al. (2001): Item-based collaborative filtering recommendation algorithms (link)
  - Hartmann (1998): A competitive genetic algorithm for resource-constrained project scheduling (link)
  - Chu and Beasley (1997): A genetic algorithm for the generalised assignment problem (link)
  - Cardoso et al. (1997): A simulated annealing approach to the solution of MINLP problems (link)
  - Suppapitnarm et al. (2000): A simulated annealing algorithm for multiobjective optimization (link)

Other suitable project directions include, for example:

- large-scale graph algorithms, such as comparing exact and goal-directed/bidirectional shortest-path methods and studying how graph size and sparsity affect performance;
- exact versus approximation algorithms for combinatorial problems such as Set Cover, Knapsack, or related problems;
- dynamic programming and memory-efficient or approximate variants, with explicit analysis of time–space trade-offs;
- randomized algorithms, such as studying success probability, repeated runs, and computational effort for randomized graph or combinatorial algorithms;
- streaming/sketching algorithms, with emphasis on accuracy–memory–running-time trade-offs; or
- exact versus approximate nearest-neighbor search, with emphasis on query time, memory, scalability, and approximation/recall quality.

These are examples only; other topics are welcome if they have sufficient algorithmic depth and are approved in the proposal.

For all project paths, the computational experiments should be designed to support meaningful conclusions rather than merely produce a collection of numerical results. Depending on the project, this may involve testing multiple problem sizes or instance characteristics, using appropriate train/test splits, repeating randomized experiments, reporting variability, measuring running time and/or memory use, and keeping comparison conditions consistent across algorithms. Whenever possible, design the experiments so that they directly answer the main questions stated in the proposal.

Submission Requirements and Final Deliverables

All submissions must be made via D2L. Project submission folders are under **Assignments** in the **Assessment** tab. For a group project, submit one copy of each required file per group and make sure that the names of all group members are clearly listed in the proposal and final report.

The final project has two deliverables. Not following the submission requirements may result in point deductions.

- **PDF report**, prepared using LaTeX or Microsoft Word/Google Docs, named DS8001_project_report_lastName1_lastName2.pdf.
  - For an individual project, omit `_lastName2`. Use the same ordering of group members in all submitted filenames.
  - If the report is prepared in Word/Google Docs, convert it to PDF before submission.
- **Zip file containing all code and documentation**, named DS8001_project_codes_lastName1_lastName2.zip.
  - The final, runnable implementation should be provided in Python files (`.py`). Jupyter notebooks may be used during development or exploratory analysis, but a jupyter notebook cannot be the submitted implementation.
  - You **must** use Python version 3.11 or later. Your final submission should run under a current Python version; points may be deducted if the code relies on behavior specific to an older unsupported version.
  - Use of standard libraries and optimization solvers is permitted when appropriate. Publicly available code may be consulted or used only when it is clearly identified and properly cited; externally sourced code does not count as your own implementation. Simply calling off-the-shelf implementations is not sufficient for the implementation component of the project. Your proposal and report should make clear which parts you implemented yourselves. Solver/library implementations may be used as comparison baselines when appropriate.
  - Include a file named `README.md`. This file should explain how to reproduce the computational experiments, including how to run the code (e.g., the main file(s), command-line arguments or other required inputs), the required libraries and relevant versions, the expected Python version, and how to obtain or generate the required data/instances if they are not included in the zip file. Where relevant, also state any random seeds or configuration files needed to reproduce the reported results. You can find information on preparing Markdown files online, e.g., see
    - `https://medium.com/@saumya.ranjan/how-to-write-a-readme-md-file-markdown-file-20cb7cbcd6f`
    - `https://github.com/awslabs/gluon-ts` – an example of a well-prepared README file in a GitHub repository.
  - Code organization, readability, appropriate documentation/commenting, and reproducibility are part of the implementation quality and will be reflected in the grade. All experimentation or temporary debugging code should be clearly separated from the final runnable workflow.

The one-page proposal should also be submitted via D2L as a PDF. A recommended filename is DS8001_project_proposal_lastName1_lastName2.pdf, with `_lastName2` omitted for an individual project.

Report Format

Your written report should contain everything needed to understand the study, your implementation, and the experimentation process. There is no page minimum or maximum; write enough to thoroughly describe the work, and *no more*. The report is also graded on clarity and conciseness. At the very least, your report should contain the following information (the section outline is provided as an example, but is highly recommended):

Section 1. **Introduction**: Explain the computational/research problem in a detailed manner. Clearly state the main question(s) that your project investigates. Aim to answer questions such as

- What is the motivation and background for the problem?
- Why are the selected algorithm(s) appropriate for the problem and for the questions being studied?

Section 2. **Literature review**: Summarize the **relevant** literature, including recent studies and important foundational work where appropriate. A literature review is expected regardless of the project type, including for an *empirical analysis-focused* project. Explain how your project relates to the existing literature rather than presenting a disconnected list of papers.

Section 3. **Methods**: Describe the underlying methodology in your own words. Explain the main algorithms precisely enough that a technically knowledgeable reader can understand how they work. Discuss relevant computational complexity and other theoretical properties when applicable. Clearly distinguish between components you implemented yourselves, components taken or adapted from external sources, and library/solver implementations used as baselines. If you proposed any modifications to an original study, describe them explicitly and explain their motivation. Also disclose data sources: if the data are publicly available, provide references; if not, explain how the synthetic data/instances were generated.

Section 4. **Experimental setup**: Provide detailed information about the experiments, including the datasets and instances, problem sizes and relevant instance characteristics, evaluation metrics, computational environment (including relevant hardware and software information), Python and library versions, and parameter and hyperparameter settings. If hyperparameter tuning is performed, explain the tuning procedure and the values considered. For randomized algorithms or experiments, report the number of repeated runs and how randomness was controlled (e.g., random seeds), where applicable. Ensure that algorithms are compared under consistent and fair experimental conditions.

Section 5. **Results**: Provide a detailed comparison and interpretation of the results. For replication projects, explicitly compare your results with those of the original study where possible and discuss meaningful discrepancies. As appropriate for the problem, examine not only solution/prediction quality but also computational performance such as running time, memory requirements, or scalability. Relate empirical behavior to theoretical expectations where possible. Distinguish observations supported by the experiments from conjectures about why an algorithm behaves in a particular way. Use plots and tables whenever appropriate, and make sure they are clear, readable, properly captioned, and referenced in the text.

Section 6. **Conclusions**: Provide the main insights from your project. Aim to answer questions such as

- What is the main takeaway from the study and the project?
- What are the limitations of the study and your project? Are there modeling, data, implementation, or experimental assumptions that may affect the conclusions? What are the main threats to validity?
- What are promising directions for further investigation?

For a group project, include a brief **contribution statement** at the end of the report describing the main contributions of each group member. Joint work is expected, but the statement should make the division of responsibilities transparent.

Tentative Grading Criteria

The project will be evaluated primarily on the quality and depth of the algorithmic investigation rather than on whether a particular algorithm achieves the best numerical performance on the chosen problem or dataset.

Criterion	Weight	What will be assessed
Proposal	5%	Clarity, feasibility, and algorithmic focus of the one-page project proposal; appropriateness of the proposed methods, instances/data, and experimental plan.
Problem & data/instances	10%	Clarity and relevance of the problem and research question(s); suitability and understanding of the datasets or problem instances; appropriate preprocessing or instance generation; awareness of important data, modeling, or instance limitations.
Methods	25%	Correct and well-motivated selection and implementation of algorithms; substantive algorithmic work by the student(s); suitable baselines/comparisons; understanding of assumptions, parameters, computational complexity, and relevant theoretical properties; methodological depth rather than mechanical use of off-the-shelf implementations.
Experiments	25%	Rigor of the experimental design; fair and controlled comparisons; appropriate datasets/instances, problem sizes, metrics, and parameter settings; suitable data splitting or tuning protocols when applicable; repetitions and stability checks for randomized methods; evaluation of computational performance such as running time, memory use, and scalability when relevant; reproducibility.
Analysis	20%	Depth of interpretation; explanation of why algorithms behave differently; connection between empirical results, theoretical expectations, and course concepts; diagnostic or sensitivity analyses where appropriate; discussion of uncertainty, limitations, unexpected results, and replication discrepancies where relevant.
Communication	15%	Organization, clarity, concision, figures/tables, technical writing, citations, code organization/documentation, and quality of the submitted reproducibility documentation (including the README).

The weights above sum to 100%. Meeting the submission requirements is part of the relevant grading criteria; for example, code correctness, organization, documentation, and reproducibility are reflected primarily under Methods, Experiments, and Communication. Failure to follow the stated submission requirements may also result in mark deductions as described above.

References

MF Cardoso, RL Salcedo, S Feyo De Azevedo, and D Barbosa. A simulated annealing approach to the solution of minlp problems. *Computers & chemical engineering*, 21(12):1349–1364, 1997.

Paul C Chu and John E Beasley. A genetic algorithm for the generalised assignment problem. *Computers & Operations Research*, 24(1):17–23, 1997.

Mukund Deshpande and George Karypis. Item-based top-n recommendation algorithms. *ACM Transactions on Information Systems (TOIS)*, 22(1):143–177, 2004.

Sönke Hartmann. A competitive genetic algorithm for resource-constrained project scheduling. *Naval Research Logistics (NRL)*, 45(7):733–750, 1998.

Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th international conference on World Wide Web*, pages 285–295, 2001.

A Suppapitnarm, Keith A Seffen, Geoff T Parks, and PJ Clarkson. A simulated annealing algorithm for multiobjective optimization. *Engineering optimization*, 33(1):59–85, 2000.